DAY 17 | Services & Networking

90 Days Challenge | Week 3: Kubernetes | Discover • Connect • Diagnose • Fix

LEVEL	TIME	FOCUS
Core skills	3–4 hours	ClusterIP, service DNS, EndpointSlices, load balancing, selector troubleshooting

GOAL OF THE DAY

Give an application a stable in-cluster address, call it by DNS name from another Pod, observe requests reaching different replicas, and diagnose why a Service stops routing when its selector is wrong.

YOUR TASKS TODAY

- Create a three-replica test workload specifically for Service networking.
- Expose it internally with a ClusterIP Service and inspect its EndpointSlices.
- Call the Service by DNS name from a temporary Pod; observe backend Pod names.
- Delete one backend Pod and observe endpoint updates.
- Break the Service selector, prove endpoints disappear, then fix it.
- Explain port, targetPort, Service DNS and why clients should use Service names rather than Pod IPs.

DELIVERABLE

A GitHub folder named day17-services/ containing the Service manifest, README, traffic-flow diagram, and evidence of DNS requests, endpoint changes, load balancing, and selector failure/fix.

SCOPE RULE — READ BEFORE STARTING

Day 15 covered basic cluster setup, an Nginx Deployment and Service YAML, scaling, and basic NodePort/port-forward access. Day 16 covered rolling updates, rollout history, failed image updates and rollback. These are not repeated. This assignment focuses on new Day 3 PPT material: ClusterIP service discovery, DNS, EndpointSlices, traffic distribution and selector troubleshooting. NodePort/port-forward is intentionally excluded.

Safety note

Use your existing local kind cluster. The task creates only a test workload named hello and its Service; do not run these commands in production.

01 / UNDERSTAND SERVICE NETWORKING

CONCEPT	MEANING / USE
Service	Stable virtual address and DNS name for matching Pods; Pod IPs can change.
ClusterIP	Default Service type, reachable inside the cluster.
Selector	Label query used to choose backend Pods; a mismatch can leave no endpoints.
EndpointSlice	Records backend addresses and ports associated with a Service.
CoreDNS	Resolves Service names; use http://hello instead of hard-coded Pod IPs.
port / targetPort	Service-facing port / port the application listens on.

Traffic path

Temporary client Pod → CoreDNS resolves hello → ClusterIP Service → cluster networking rules → a Ready backend Pod on port 8080.

Command reference

COMMAND	PURPOSE
kubectl get svc -o wide	View Service type, ClusterIP, ports and selector.
kubectl get pods --show-labels	Compare Pod labels with the Service selector.
kubectl get endpointslices -l kubernetes.io/service-name=hello	Inspect backend addresses and ports.
kubectl describe svc hello	Inspect selector, ports and endpoints.
kubectl run tmp --rm -it --image=busybox:1.36 --restart=Never -- sh	Start a temporary in-cluster client.
nslookup hello.default.svc.cluster.local	Resolve full Service DNS name from inside the cluster.
kubectl get events --sort-by=.metadata.creationTimestamp	Review recent cluster events.

Ingress, Helm, NetworkPolicy and NodePort/port-forward tasks are out of scope for this assignment.

02 / CREATE THE SERVICE & TEST DNS

Step 1 — Check your current cluster

```
kubectl config current-context
kubectl get nodes
kubectl get deployments
kubectl get services
```

Reuse your existing cluster; do not rebuild it.

Step 2 — Create a three-replica test workload

```
kubectl create deployment hello --replicas=3 \
  --image=registry.k8s.io/e2e-test-images/agnhost:2.39 \
  -- /agnhost netexec --http-port=8080

kubectl get deployment hello
kubectl get pods -l app=hello --show-labels
```

Wait until all three Pods are Ready.

Step 3 — Create a ClusterIP Service

```
kubectl expose deployment hello --name=hello \
  --type=ClusterIP --port=80 --target-port=8080

kubectl get svc hello -o wide
kubectl describe svc hello
kubectl get endpointslices \
  -l kubernetes.io/service-name=hello
```

Manifest alternative — svc-clusterip.yaml

```
apiVersion: v1
kind: Service
metadata:
  name: hello
spec:
  type: ClusterIP
  selector:
    app: hello
  ports:
    - port: 80
      targetPort: 8080
```

Use either kubectl expose or the manifest—not both. If using the manifest, apply it with kubectl apply -f svc-clusterip.yaml.

Step 4 — Call the Service by name

```
kubectl run tmp --rm -it --image=busybox:1.36 \
  --restart=Never -- sh
```

Inside the temporary shell:

```
nslookup hello
nslookup hello.default.svc.cluster.local
for i in 1 2 3 4 5 6; do
  wget -qO- http://hello/hostname
  echo
done
exit
```

Expected: requests return backend hostnames. With three Ready replicas, repeated requests should reach more than one backend over the series; exact order is not guaranteed.

> If DNS resolves but requests fail, check targetPort, Pod readiness and EndpointSlices before changing unrelated settings.

03 / WATCH ENDPOINTS & BREAK/FIX

Step 5 — Observe backend endpoints

```
kubectl get endpointslices \
  -l kubernetes.io/service-name=hello -o wide
kubectl get pods -l app=hello -o wide
```

Record the backend addresses and confirm they correspond to Ready Pods selected by the Service.

Step 6 — Delete one backend Pod and observe replacement

```
kubectl get pods -l app=hello
kubectl delete pod POD_NAME
kubectl get pods -l app=hello -w
```

Replace POD_NAME with one Pod from this workload. Wait for a replacement, then press Ctrl+C.

```
kubectl get endpointslices \
  -l kubernetes.io/service-name=hello -o wide
kubectl get pods -l app=hello
```

Capture evidence that the endpoint set changes and the workload returns to three Ready replicas.

Step 7 — Break the selector

```
kubectl patch service hello \
  -p '{"spec":{"selector":{"app":"hellow"}}}'

kubectl describe service hello
kubectl get endpointslices \
  -l kubernetes.io/service-name=hello
```

Expected: the selector no longer matches app=hello Pods, so backend endpoints become empty. DNS may still resolve because the Service exists, but requests cannot reach an application backend.

Step 8 — Fix and verify

```
kubectl patch service hello \
  -p '{"spec":{"selector":{"app":"hello"}}}'

kubectl get endpointslices \
  -l kubernetes.io/service-name=hello -o wide
kubectl describe service hello
```

Run the BusyBox request loop again and confirm requests succeed.

> Check the Service selector, Pod labels and EndpointSlices. The Service object existing does not prove it has a usable backend.

04 / TROUBLESHOOTING & SUBMISSION

PROBLEM	CHECK / FIX
No endpoints	Compare kubectl describe svc hello with kubectl get pods --show-labels; check selector and readiness.
DNS lookup fails	Run nslookup inside the cluster; check Service name, namespace and CoreDNS.
DNS resolves but request fails	Check EndpointSlices, targetPort 8080, Pod readiness and app port.
Only one backend appears	Ensure all three Pods are Ready and repeat requests; traffic order is not guaranteed.
Pod replacement takes time	Use kubectl get pods -w and inspect events until replacement is Ready.
ClusterIP unreachable from laptop	Expected: ClusterIP is internal. Test from a Pod; this is not a laptop port-forward task.

Definition of done

- Service hello is ClusterIP and its selector matches app=hello.
- EndpointSlices show three Ready backend addresses when healthy.
- DNS and repeated requests work from a temporary Pod.
- Deleting a backend Pod changes endpoints and a replacement becomes Ready.
- Wrong selector makes endpoints empty; fixing it restores requests.
- README explains port vs targetPort, Service DNS and EndpointSlices.

Submission checklist

- Include svc-clusterip.yaml, README.md, and evidence screenshots/terminal output.
- Show DNS lookup, several backend hostnames, and EndpointSlices before/after changes.
- Include selector failure/fix and a traffic-flow diagram.
- Do not commit kubeconfig, credentials or tokens.

Suggested repository structure

```
day17-services/
  svc-clusterip.yaml
  README.md
  architecture.png
  evidence/
    service-endpoints.png
    dns-load-balancing.png
    pod-replacement.png
    selector-failure-fix.png
```

Cleanup

```
kubectl delete service hello
kubectl delete deployment hello
```

Only delete resources created for this assignment; leave your shared cluster and unrelated resources intact.

05 / INTERVIEW READINESS

QUESTION	KEY IDEA
Why use a Service instead of a Pod IP?	Pod IPs can change; a Service gives a stable virtual IP and DNS name.
ClusterIP vs NodePort vs LoadBalancer?	ClusterIP is internal; NodePort opens a port on each node; LoadBalancer requests an external load balancer where supported.
port vs targetPort?	port is the Service port; targetPort is the backend application port.
How does a Pod find a Service by name?	CoreDNS resolves the Service name to its ClusterIP.
What is an EndpointSlice?	A resource that records network endpoints associated with a Service.
Why can a Service have no endpoints?	The selector matches no Ready Pods, labels differ, or backend readiness/port configuration is wrong.
Why can DNS work while HTTP fails?	The Service may exist but have no usable backend endpoints or the targetPort may be wrong.

BONUS CHALLENGE

Create a second temporary client Pod and call http://hello/hostname. Explain why both clients can use the same Service DNS name without knowing any Pod IPs.

REVIEW CRITERIA (100 POINTS)

AREA	POINTS
ClusterIP Service and correct selector/ports	20
DNS lookup and repeated requests from a Pod	20
EndpointSlice inspection and Pod replacement evidence	20
Selector failure diagnosed and fixed	25
README, diagram, screenshots and cleanup	15

FINAL DELIVERABLE: Submit the GitHub repository link and evidence from your own cluster. Do not fabricate endpoint addresses, hostnames or successful command output.